Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 6
Dynamic Programming (Part 1)
Δυναμικός Προγραμματισμός (Μέρος 1)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

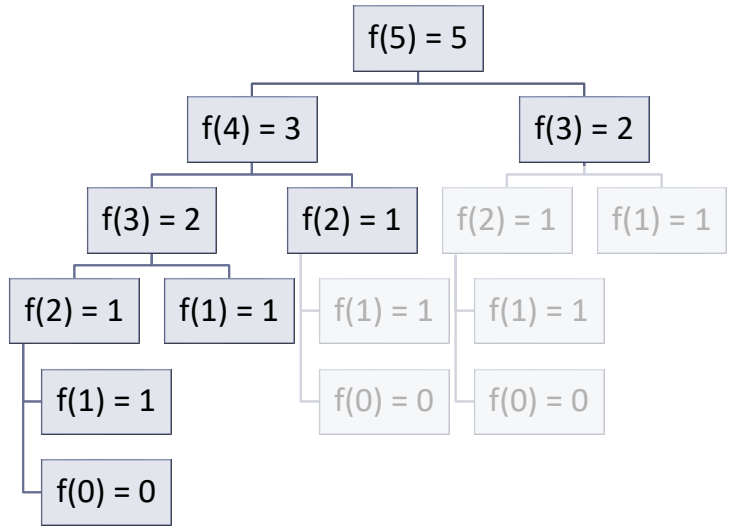
1

Dynamic Programming – What is it?

- **Dynamic programming** (**Δυναμικός προγραμματισμός**), DP for short, is a technique similar to divide-and-conquer
  - They both solve problems by combining solutions to subproblems
- However, DP
  - solves subproblems first, and
  - saves the solutions to subproblems for later use
- Thus, DP is applicable when the subproblems are not independent
  - i.e., when subproblems share subsubproblems
- *Note*: "programming" does not refer to writing code, it simply refers to a tabular method (μέθοδος πινάκων)

2

Computing the Fibonacci Sequence



Optimization Problems

- **Optimization problem** (**Πρόβλημα βελτιστοποίησης**)
  - Find a solution that minimizes or maximizes the value of some real-valued function defined on the solutions
  - *Applications*: industrial (shape of a can), shipping (use of cranes), economics (max utility), finance (optimal portfolios)
- Solutions
  - Each solution has a value and we wish to find the one with the optimal (min or max) value
  - We call such a solution an **optimal solution** (**βέλτιστη λύση**)
  - There is no guarantee that such a solution will be unique
  - There is no guarantee that you will find it (local maxima, anyone?) unless the search is over a small enough search space or the function is restricted enough

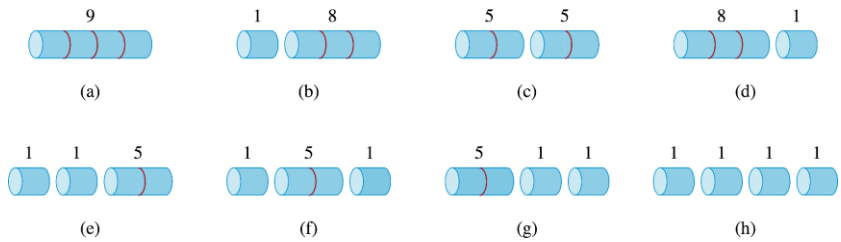
Example: Rod Cutting (Κοπή της ράβδου)

- You are given a rod (ράβδος) of length $n \geq 0$ (n in cm)
- A rod of length i cm will be sold for p_i euros
- Cutting is free (simplifying assumption)
- Problem: Given a table of prices p_i determine the maximum revenue (μέγιστο εισόδημα) r_n obtainable by cutting up the rod and selling the pieces

Length i	1	2	3	4	5	6	7	8	9	10
Price p_i	1	5	8	9	10	17	17	20	24	30

Example: Rod Cutting
Step 1: Characterizing an Optimal Solution

- **Question**: In how many different ways can we cut a rod of length n?
- For a rod of length 4:



- $2^{4-1} = 2^3 = 8$
- For a rod of length n: 2^{n-1}. **Exponential** (**Εκθετικός**): we cannot try all possibilities for "large" n. The obvious exhaustive approach won't work.

Example: Rod Cutting
Step 1: Characterizing an Optimal Solution

- **Question**: In how many different ways can we cut a rod of length n?
  - **Proof Details**: a rod of length n can have exactly $n-1$ possible cut positions – choose $0 \leq k \leq n-1$ actual cuts. We can choose the k cuts (without repetition) anywhere we want, so that for each such k the number of different choices is
$$\binom{n-1}{k}$$
  - When we sum up over all possibilities ($k = 0$ to $k = n-1$):
$$\sum_{k=0}^{n-1} \binom{n-1}{k} = \sum_{k=0}^{n-1} \frac{(n-1)!}{k!\,(n-1-k)!} = (1+1)^{n-1} = 2^{n-1}$$
- For a rod of length n: 2^{n-1} cuts

Example: Rod Cutting
Step 1: Characterizing an Optimal Solution

- Let us find a way to solve the problem **recursively** (**αναδρομικά**)
- Assume we have cut a rod of length n into $1 \leq k \leq n$ pieces of length $i_1, \ldots, i_k$:
$$n = i_1 + \cdots + i_k$$
with revenue
$$r_n = p_{i_1} + \cdots + p_{i_k}$$
- If this solution is optimal, how can we construct it?
- **Advice:** when you don't know what to do next, start with a simple example and hope something will occur to you…

Example: Rod Cutting

Step 1: Characterizing an Optimal Solution

Length i	1	2	3	4	5	6	7	8	9	10
Price p_i	1	5	8	9	10	17	17	20	24	30

- We begin by constructing (by hand) the optimal solutions for $i = 1, \ldots, 10$:
  - $r_1 = 1$ from solution 1 = 1 (no cuts)
  - $r_2 = 5$ from solution 2 = 2 (no cuts)
  - $r_3 = 8$ from solution 3 = 3 (no cuts)
  - $r_4 = 10$ from solution 4 = 2 + 2
  - $r_5 = 13$ from solution 5 = 2 + 3
  - $r_6 = 17$ from solution 6 = 6 (no cuts)
  - $r_7 = 18$ from solution 7 = 1 + 6 or 7 = 2 + 2 + 3
  - $r_8 = 22$ from solution 8 = 2 + 6
  - $r_9 = 25$ from solution 9 = 3 + 6
  - $r_{10} = 30$ from solution 10 = 10 (no cuts)

Example: Rod Cutting

Step 2: Recursive Optimal Solution

- In some cases $r_n = p_n$, while in other cases the optimal revenue r_n is obtained by cutting the rod into smaller pieces.
- In **ALL** cases we have the recursion

  $r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \ldots, r_{n-1} + r_1)$
- exhibiting **optimal substructure** (**βέλτιστη υποδομή**)
  - Optimal solutions to a problem incorporate (ενσωματώνουν) optimal solutions to related subproblems
- A slightly different way of stating the same recursion, which avoids repeating some computations, is

  $r_n = \max_{1 \le i \le n} (p_i + r_{n-i})$

```
CUT-ROD(p, n)
1  if n == 0
2      return 0
3  q = -∞
4  for i = 1 to n
5      q = max(q, p[i] + CUT-ROD(p, n - i))
6  return q
```

Example: Rod Cutting
Step 2: Recursive Optimal Solution

- How to justify the step from:

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1)$$

to

$$r_n = \max_{1 \le i \le n} (p_i + r_{n-1})$$

- **Note**: every optimal partitioning of a rod of length n has a first cut – a segment of, say, length i. The optimal revenue, r_n, must satisfy $r_n = p_i + r_{n-i}$, where r_{n-i} is the optimal revenue for a rod of length $n - i$. If the latter were not the case, there would be a better partitioning for a rod of length $n - i$, giving a revenue $r'_{n-i} > r_{n-i}$ and a total revenue $r'_n = p_i + r'_{n-i} > p_i + r_{n-i} = r_n$
- Since we do not know which one of the leftmost cut positions provides the largest revenue, we just maximize over all the possible first cut positions.

Example: Rod Cutting
Step 1: Characterizing an Optimal Solution

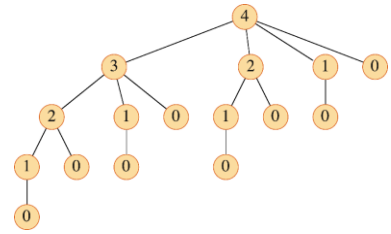
- We can also notice that all the items we choose the maximum of are optimal in their own right: each substructure (max revenue for rods of lengths $1, \dots, n-1$) is also optimal (again, **optimal substructure property**).

- Nevertheless, we are still in trouble: computing the recursion leads to recomputing a number (= overlapping subproblems) of values – how many?

Example: Rod Cutting
Step 1: Characterizing an Optimal Solution

Let's call Cut-Rod(p, 4), to see the effects on a simple case:

```
CUT-ROD(p, n)
1  if n == 0
2      return 0
3  q = -∞
4  for i = 1 to n
5      q = max(q, p[i] + CUT-ROD(p, n - i))
6  return q
```



Let $T(n)$ denote the total number of calls made to Cut-Rod(p, n)
The number of nodes for a tree corresponding to a rod of size n is:

$$T(0)=1, \quad T(n)=1+\sum_{j=0}^{n-1} T(j)=2^n, n \geq 1.$$

Example: Rod Cutting
Beyond Naïve Time Complexity

- Observation:
  - The same subproblems are solved repeatedly!
- Idea:
  - Save the values associated with each $T(j)$, so that we compute each value only once (called "**memoization** (**υπομνηματισμός**)" = writing yourself a memo)
- **Time-memory trade-off**
  - Use additional memory to save computation time
  - Savings may be dramatic: from exponential to polynomial

Example: Rod Cutting
Naïve Caching – Top-down with Memoization

- We introduce two procedures:

```
MEMOIZED-CUT-ROD(p, n)
1  let r[0..n] be a new array
2  for i = 0 to n
3      r[i] = -∞
4  return MEMOIZED-CUT-ROD-AUX(p, n, r)

MEMOIZED-CUT-ROD-AUX(p, n, r)
1  if r[n] ≥ 0
2      return r[n]
3  if n == 0
4      q = 0
5  else q = -∞
6      for i = 1 to n
7          q = max(q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n - i, r))
8  r[n] = q
9  return q
```


Example: Rod Cutting
Step 3: *Non-recursive Optimal Solution*

- We now remove some unnecessary complications and write the method bottom-up (i.e., **sort the subproblems by size and solve them in size order, smallest first**):

```
BOTTOM-UP-CUT-ROD(p, n)
1  let r[0..n] be a new array
2  r[0] = 0
3  for j = 1 to n
4      q = -∞
5      for i = 1 to j
6          q = max(q, p[i] + r[j - i])
7      r[j] = q
8  return r[n]
```

Example: Rod Cutting
Time Complexity

- Whether we solve the problem in a top-down or bottom-up manner the asymptotic time is $\Theta(n^2)$, the major difference being recursive calls as compared to loop iterations.
- Why?

Example: Rod Cutting
Step 4: *Reconstructing a Solution*

- Extend the DP approach to record not only the optimal *value* for each subproblem, but also a *choice* that led to the optimal value

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
 1  let r[0..n] and s[0..n] be new arrays
 2  r[0] = 0
 3  for j = 1 to n
 4      q = -∞
 5      for i = 1 to j
 6          if q < p[i] + r[j - i]
 7              q = p[i] + r[j - i]
 8              s[j] = i
 9      r[j] = q
10  return r and s
```

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

Dynamic Programming - Steps

1. Characterize the structure of an optimal solution
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution in a bottom-up fashion
4. Construct an optimal solution from computed information

- Step 4 may be omitted, if only the optimal value is required (and not the actual solution)

Dynamic Programming - DP requirements

Two requirements:

1. **Overlapping subproblems** (**Επικαλυπτόμενα υποπροβλήματα**)
   - A recursive algorithm for the problem would solve the same subproblems over and over
   - Dynamic-programming instead solves each subproblem once and then stores the solution in a table where it can be looked up when needed
2. **Optimal substructure** (**Βέλτιστη υποδομή**)
   - A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems
   - In dynamic programming, we build an optimal solution to the problem from optimal solutions to subproblems

Divide & Conquer VS Dynamic Programming

	Divide & Conquer	Dynamic Programming
View problem as collection of subproblems	✓	✓
"Recursive" nature	✓	✓
Independent subproblems (Ανεξάρτητα υποπροβλήματα)	✓	
Overlapping subproblems (Επικαλυπτικά υποπροβλήματα)		✓
Number of subproblems	depends on partitioning factors	typically small
Characteristic running time	typically log function of n	depends on number and difficulty of subproblems
Primarily for optimization problems (προβλήματα βελτιστοποίησης)		✓
Optimal substructure (Βέλτιστη υποδομή)		✓